ASSIGNMENT 6

Automate Network Deployment

Terraform + AWS — Complete Step-by-Step Student Guide

Tools	Terraform CLI + AWS CLI + AWS Management Console
Platform	Windows PowerShell
Practical Task	Create VPC + Subnet → Deploy → Modify → Redeploy
Learning Outcome	Understand Infrastructure as Code (IaC) and automated network deployment

Prepared as a reusable lab guide for students

1. Practical Objective

In this practical, students use Infrastructure as Code (IaC) to automate the creation and modification of an AWS network. Terraform describes the desired infrastructure in code and then creates or updates the corresponding AWS resources.

- Install Terraform on Windows.
- Install AWS CLI on Windows.
- Create an AWS IAM user for lab automation and generate an access key.
- Configure AWS CLI credentials.
- Create a Terraform working folder using mkdir.
- Create main.tf using Notepad.
- Write Terraform code for a VPC and subnet.
- Run terraform init, terraform plan, and terraform apply.
- Verify the VPC and subnet in the AWS Console and using AWS CLI.
- Modify the VPC configuration and redeploy using terraform apply.

2. Important Terms — Learn These First

Term	Meaning
AWS	Amazon Web Services — the cloud platform where the VPC and subnet are created.
VPC	Virtual Private Cloud — a logically isolated virtual network inside AWS.
Subnet	A smaller network segment inside a VPC.
Terraform	An Infrastructure as Code tool used to define and manage cloud infrastructure using configuration files.
IaC	Infrastructure as Code — managing infrastructure through code instead of creating every resource manually in a web console.
AWS CLI	A command-line tool used to interact with AWS from PowerShell or Command Prompt.
IAM	Identity and Access Management — AWS service used to control users, permissions, and credentials.
Access Key ID	The public identifier used by AWS CLI to identify an access-key credential.
Secret Access Key	A private credential paired with the Access Key ID. Never share it or put it in a public document.
terraform plan	Shows what Terraform intends to create, change, or destroy before applying it.
terraform apply	Actually performs the infrastructure changes in AWS.

3. Install AWS CLI on Windows

Step 3.1: Download AWS CLI

Meaning: AWS CLI lets you run AWS commands from PowerShell. AWS provides an official Windows MSI installer.

```
Official AWS installation page:
https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html
```

- For a normal 64-bit Windows computer, download the AWS CLI Version 2 Windows MSI installer.
- Run the installer and complete the installation wizard.
- Close and reopen PowerShell after installation so the PATH is refreshed.

Step 3.2: Verify AWS CLI

Meaning: Confirm that the aws command is available.

```
aws --version
```

Step 3.3: Verify the active AWS identity later

Meaning: After credentials are configured, this command tells you which AWS identity the CLI is using.

```
aws sts get-caller-identity
```

4. Install Terraform on Windows

Step 4.1: Download Terraform

Meaning: Terraform is the IaC tool used to create the VPC and subnet from code.

```
Official HashiCorp installation page: https://developer.hashicorp.com/terraform/install
```

- Choose Windows and the correct architecture, normally AMD64 for a standard 64-bit Intel/AMD Windows machine.
- Download the ZIP package, extract terraform.exe, and place it in a permanent folder such as C:\Terraform.
- Add C:\Terraform to the Windows PATH if it is not already available.
- Close and reopen PowerShell.

Step 4.2: Verify Terraform

Meaning: Confirm that PowerShell can find Terraform.

```
terraform -version
```

5. Prepare AWS Credentials for Terraform

For a student lab, an IAM user can be used for CLI-based authentication when the AWS account's policies allow it. The user must have permission to create the resources required by the lab.

Step 5.1: Open IAM

Meaning: Sign in to the AWS Management Console and open IAM → Users.

- Create a user such as terraform-user.
- Grant only the permissions required by the lab/account policy. Avoid giving permanent administrator access unless your instructor/account owner explicitly requires it.

Step 5.2: Create an Access Key

Meaning: An access key is a credential pair used by AWS CLI. AWS shows the Secret Access Key only when the key is created, so save it securely.

```
IAM → Users → terraform-user → Security credentials → Access keys → Create access key
```

- For the use-case screen, select the option appropriate for command-line use (for example, Command Line Interface), or follow your account's current AWS guidance.
- Download the CSV or securely copy the Access Key ID and Secret Access Key.
- Never paste the Secret Access Key into a student handout, Git repository, screenshot, or chat.

AWS note: the Secret Access Key cannot be retrieved later. If it is lost or exposed, deactivate/delete the old key and create a new one.

6. Configure AWS CLI

Step 6.1: Run aws configure

Meaning: This stores the credentials used by AWS CLI.

```
aws configure
```

Enter the values when prompted:

```
AWS Access Key ID:     <your-access-key-id>
AWS Secret Access Key: <your-secret-access-key>
Default region name:   us-east-1
Default output format: json
```

Step 6.2: Test the credentials

Meaning: AWS STS confirms which account/user the CLI is currently using.

```
aws sts get-caller-identity
```

Expected result: JSON containing UserId, Account, and Arn. Do not publish the complete output in a public document.

7. Create the Terraform Project Folder

Step 7.1: Open PowerShell

Meaning: PowerShell is the Windows command-line environment used for this lab.

Step 7.2: Create the folder

Meaning: mkdir creates a new directory.

```
mkdir terraform-network
```

Step 7.3: Enter the folder

Meaning: cd changes the current working directory.

```
cd terraform-network
```

Step 7.4: Create main.tf

Meaning: main.tf is the main Terraform configuration file.

```
notepad main.tf
```

If Notepad asks whether to create the file, choose Yes.

8. Write the Terraform IaC Script

Paste the following beginner-friendly example into main.tf:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_vpc" "main_vpc" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "Assignment6-VPC"
  }
}

resource "aws_subnet" "main_subnet" {
  vpc_id            = aws_vpc.main_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "us-east-1a"

  tags = {
    Name = "Assignment6-Subnet"
  }
}
```

Save the file: Ctrl + S, then close Notepad.

9. Understand the Terraform Script

terraform block — Tells Terraform which provider/plugin is required.

provider "aws" — Tells Terraform that AWS is the cloud provider and selects the AWS region.

resource "aws_vpc" — Defines the VPC Terraform must manage.

cidr_block = "10.0.0.0/16" — Defines the VPC IPv4 address range.

resource "aws_subnet" — Defines the subnet inside the VPC.

vpc_id = aws_vpc.main_vpc.id — Connects the subnet to the VPC created by Terraform.

tags — Adds a readable Name to the AWS resource.

10. Initialize Terraform

Step 10.1: Run terraform init

Meaning: Downloads/initializes the AWS provider and prepares the current folder to use Terraform.

```
terraform init
```

Look for a successful message indicating that Terraform has been initialized.

11. Validate the Configuration

Step 11.1: Run terraform validate

Meaning: Checks whether the Terraform configuration is syntactically and structurally valid.

```
terraform validate
```

Step 11.2: Optional formatting

Meaning: Formats the Terraform file into the standard Terraform style.

```
terraform fmt
```

12. Preview the Deployment

Step 12.1: Run terraform plan

Meaning: Terraform compares the desired configuration with the current infrastructure and shows the proposed changes.

```
terraform plan
```

For a new VPC + subnet, a typical result is similar to:

```
Plan: 2 to add, 0 to change, 0 to destroy.
```

Do not type yes at this stage. terraform plan is only a preview.

13. Deploy the Infrastructure

Step 13.1: Run terraform apply

Meaning: Terraform now performs the planned AWS changes.

```
terraform apply
```

- Review the plan shown by Terraform.
- When Terraform asks for approval, type exactly: yes

```
Enter a value: yes
```

Successful output for this practical is similar to:

```
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

14. Verify the VPC and Subnet

Step 14.1: Verify from AWS CLI

Meaning: This lists VPCs visible to the configured identity in the selected region.

```
aws ec2 describe-vpcs --region us-east-1
```

Step 14.2: Verify from AWS Console

Meaning: Open AWS Console → VPC → Your VPCs. Find Assignment6-VPC and confirm State = Available.

Step 14.3: Verify the subnet

Meaning: Open VPC → Subnets. Find Assignment6-Subnet and confirm its VPC and CIDR block.

15. Modify and Redeploy

This is the second half of the practical. Instead of deleting and recreating the VPC, we change a property that Terraform can update in place.

Step 15.1: Modify the VPC Name tag

Meaning: Open main.tf and change the VPC Name tag.

```
Name = "Assignment6-VPC"  →  Name = "Assignment6-VPC-modified"
```

Step 15.2: Run terraform plan

Meaning: Terraform should detect one in-place update.

```
terraform plan

Plan: 0 to add, 1 to change, 0 to destroy.
```

Step 15.3: Apply the modification

Meaning: Run apply and approve the change.

```
terraform apply

Enter a value: yes
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.
```

The key learning point is that Terraform recognizes the existing VPC and updates its tag instead of creating a second VPC.

16. Common Errors and How to Understand Them

Problem	Meaning / Action
UnauthorizedOperation / explicit deny	The AWS identity does not have permission to perform the requested EC2 action, or an account/SCP/identity policy explicitly denies it. Check the IAM permissions and organizational policies. Do not try to bypass the policy.
aws: command not found / not recognized	AWS CLI is not installed correctly or the PATH has not refreshed. Reinstall/check the installation and close/reopen PowerShell.
terraform: command not found / not recognized	Terraform is not on PATH. Confirm terraform.exe exists and add its folder to PATH, then reopen PowerShell.
Wrong AWS account or identity	Run aws sts get-caller-identity and compare the account/ARN with the account you intend to use.
Access key exposed	Immediately deactivate/delete the exposed key and create a replacement. Never commit credentials to Git.

17. Optional Cleanup After the Practical

If the instructor asks students to remove the lab resources after assessment, Terraform can destroy only the resources managed by the current configuration.

```
terraform plan -destroy
terraform destroy
```

When prompted, type yes only after confirming that the resources shown are safe to delete.

Important: Do not run terraform destroy if you need the VPC/subnet for demonstration, grading, or another practical.

18. Student Submission Checklist

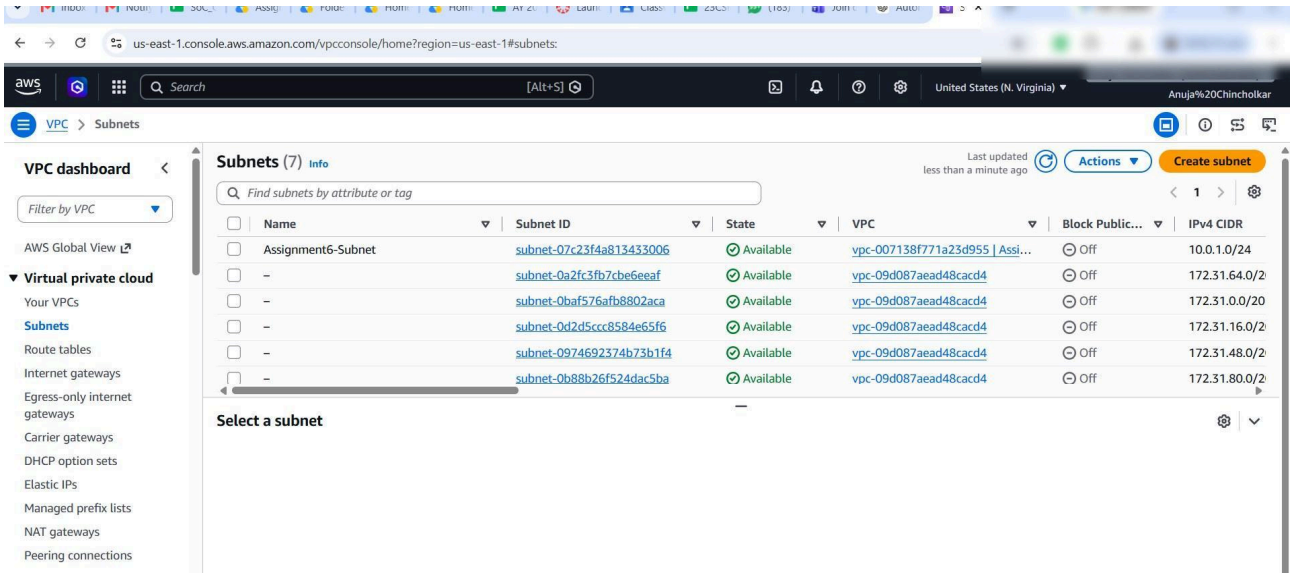
- ☐ Terraform installed and terraform -version works.

- ☐ AWS CLI installed and aws --version works.
- ☐ IAM credentials configured securely.
- ☐ aws sts get-caller-identity succeeds.
- ☐ terraform init succeeds.
- ☐ terraform validate succeeds.
- ☐ terraform plan shows the expected resources.
- ☐ terraform apply creates the VPC and subnet.
- ☐ AWS Console shows the VPC and subnet.
- ☐ A modification is made in main.tf.
- ☐ terraform plan shows 0 add, 1 change, 0 destroy (or the expected equivalent for the chosen change).
- ☐ terraform apply successfully redeploys the modification.

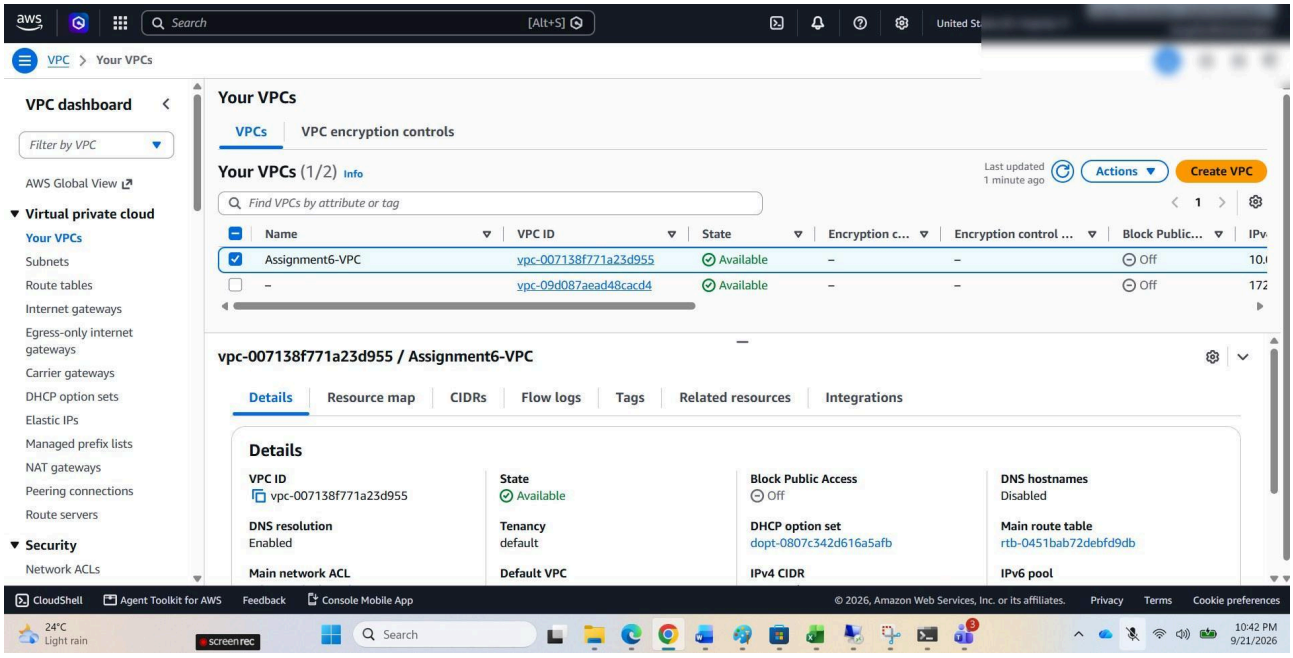
19. Lab Screenshots / Evidence

The following screenshots are included as visual references from the completed lab session. Account/profile header areas have been blurred where appropriate for safer sharing. Students should reproduce the steps using their own AWS account and credentials.

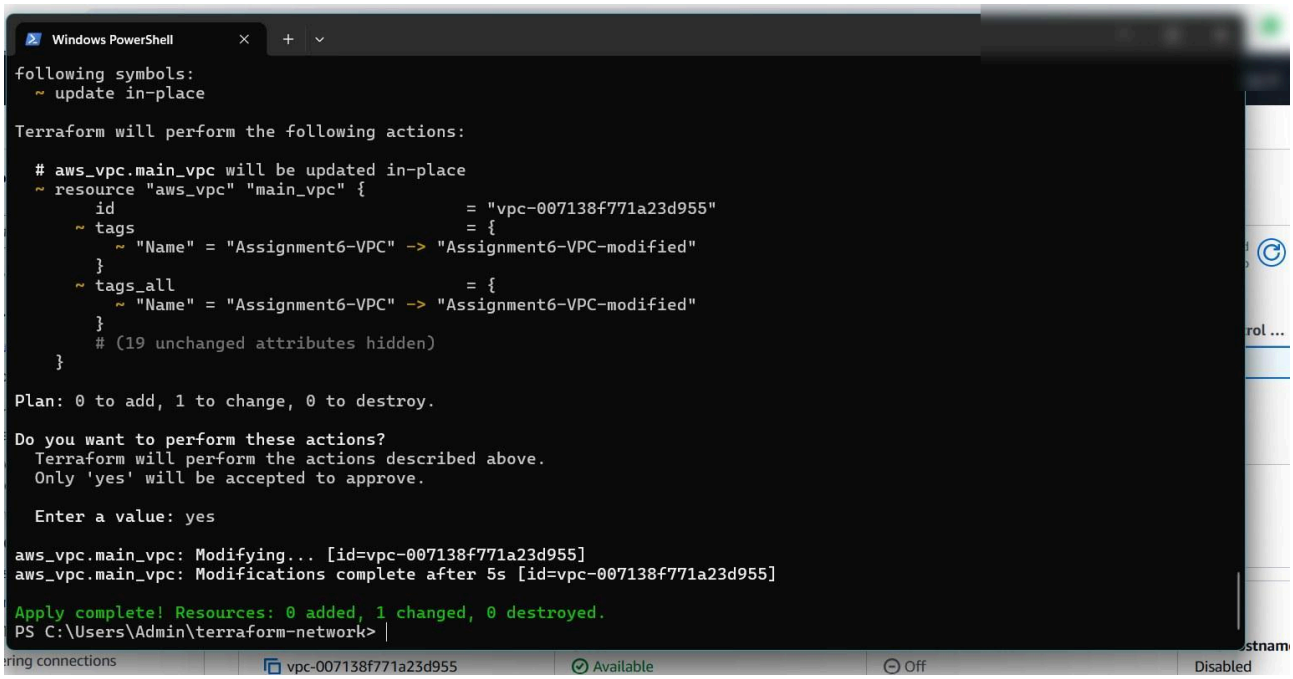
Screenshot 1



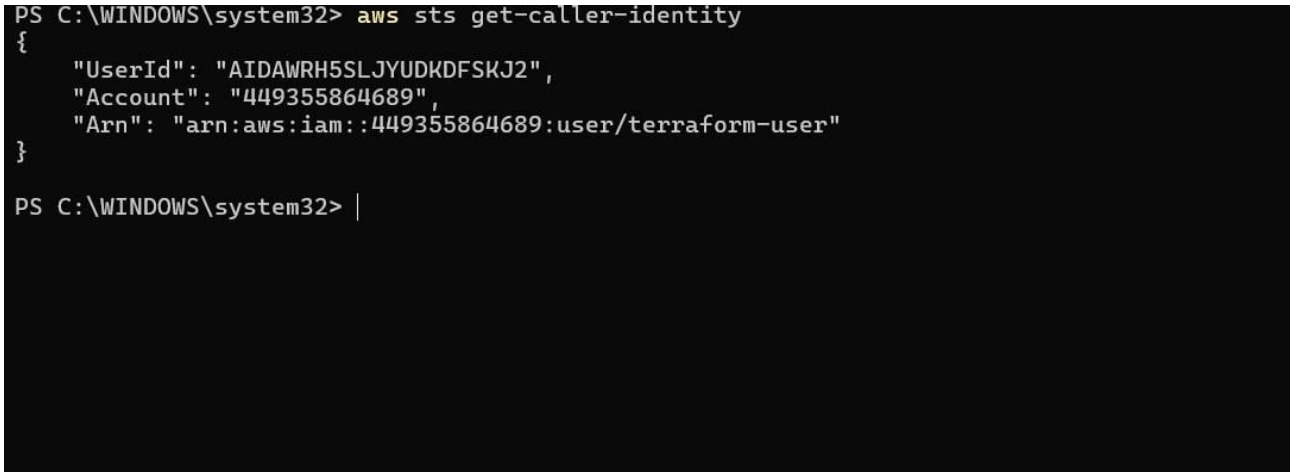
Screenshot 2



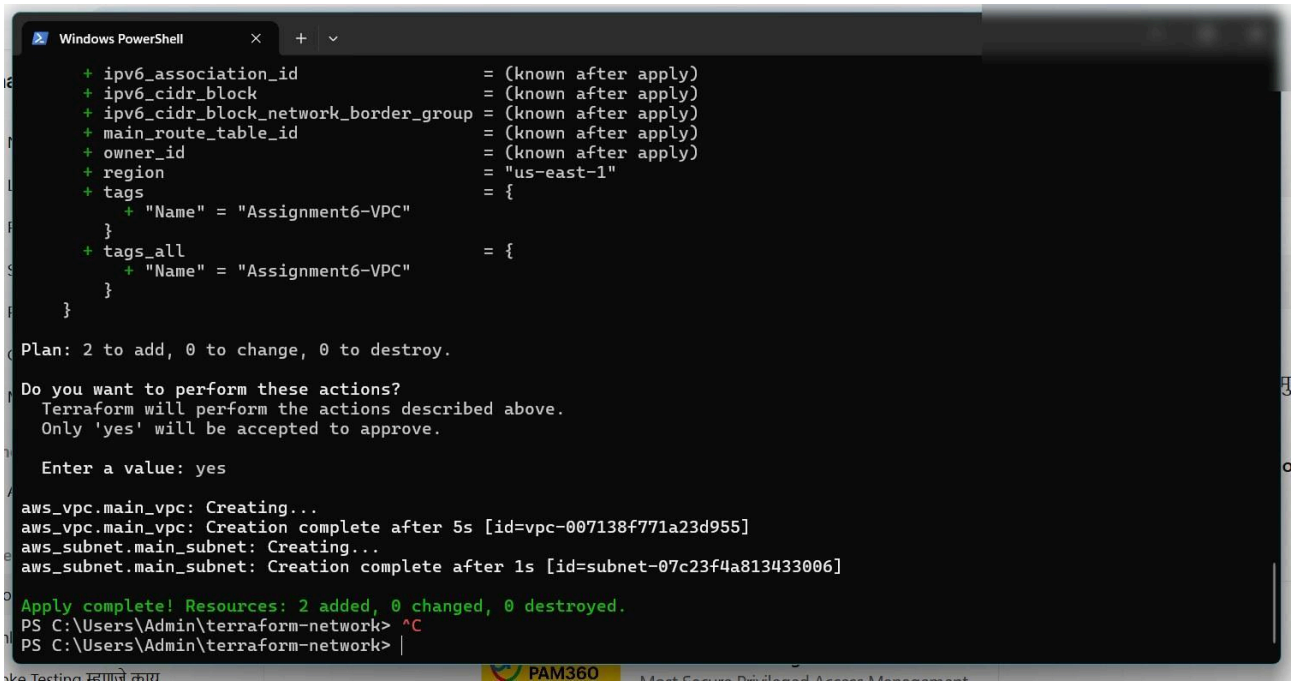
Screenshot 3



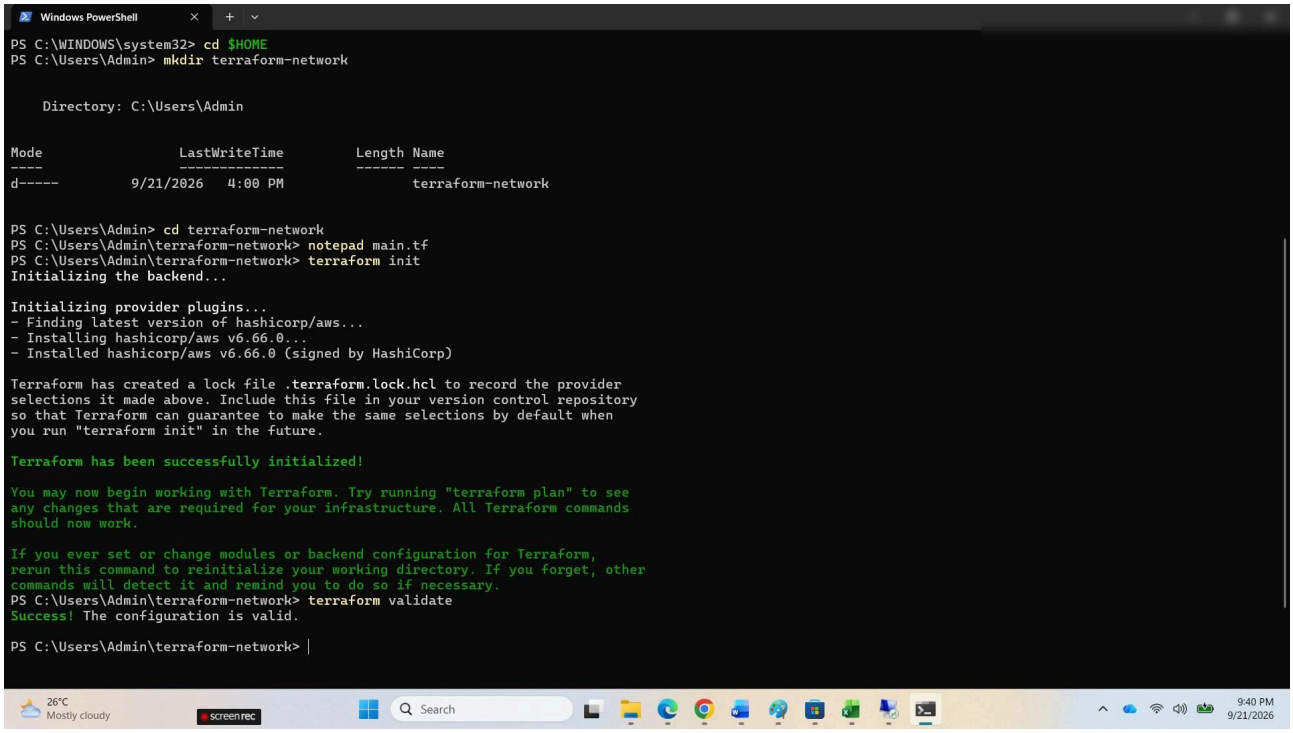
Screenshot 4



Screenshot 5



Screenshot 6



Screenshot 7

```
PS C:\Users\Admin> cd terraform-network
PS C:\Users\Admin\terraform-network> notepad main.tf
PS C:\Users\Admin\terraform-network> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.66.0...
- Installed hashicorp/aws v6.66.0 (signed by HashiCorp)

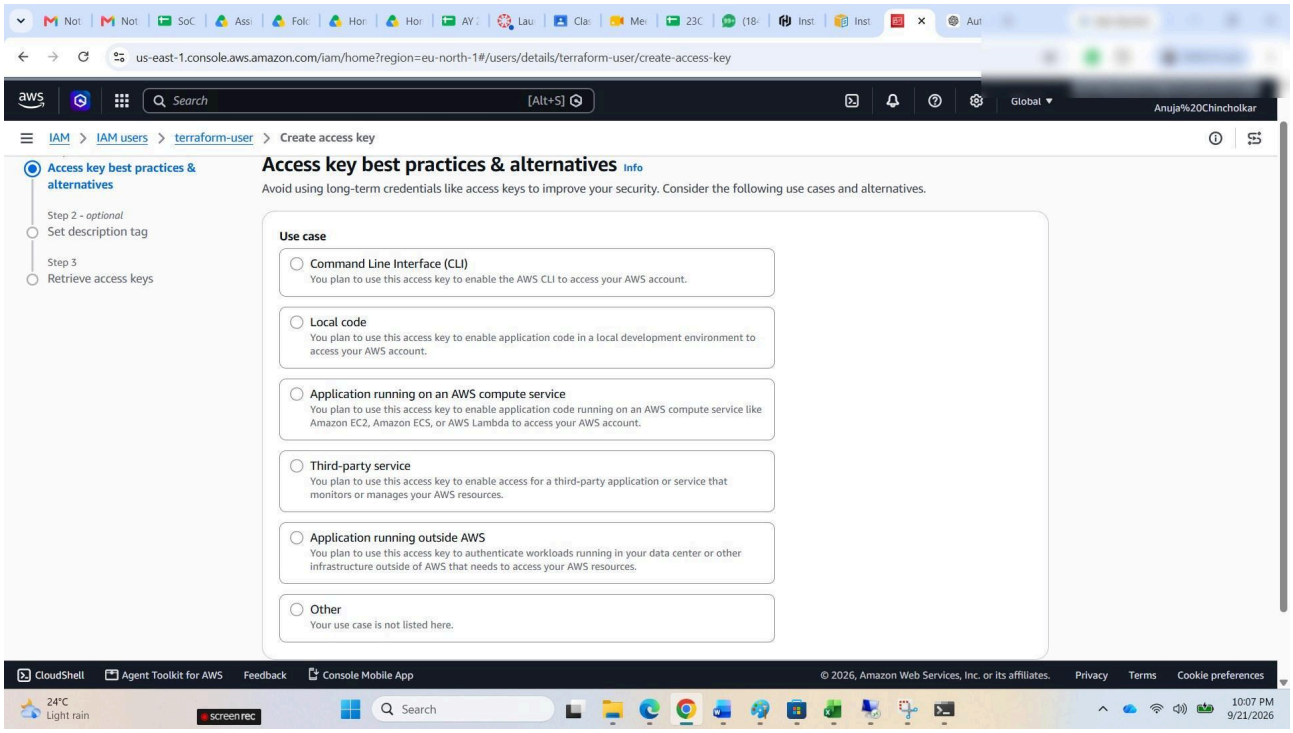
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

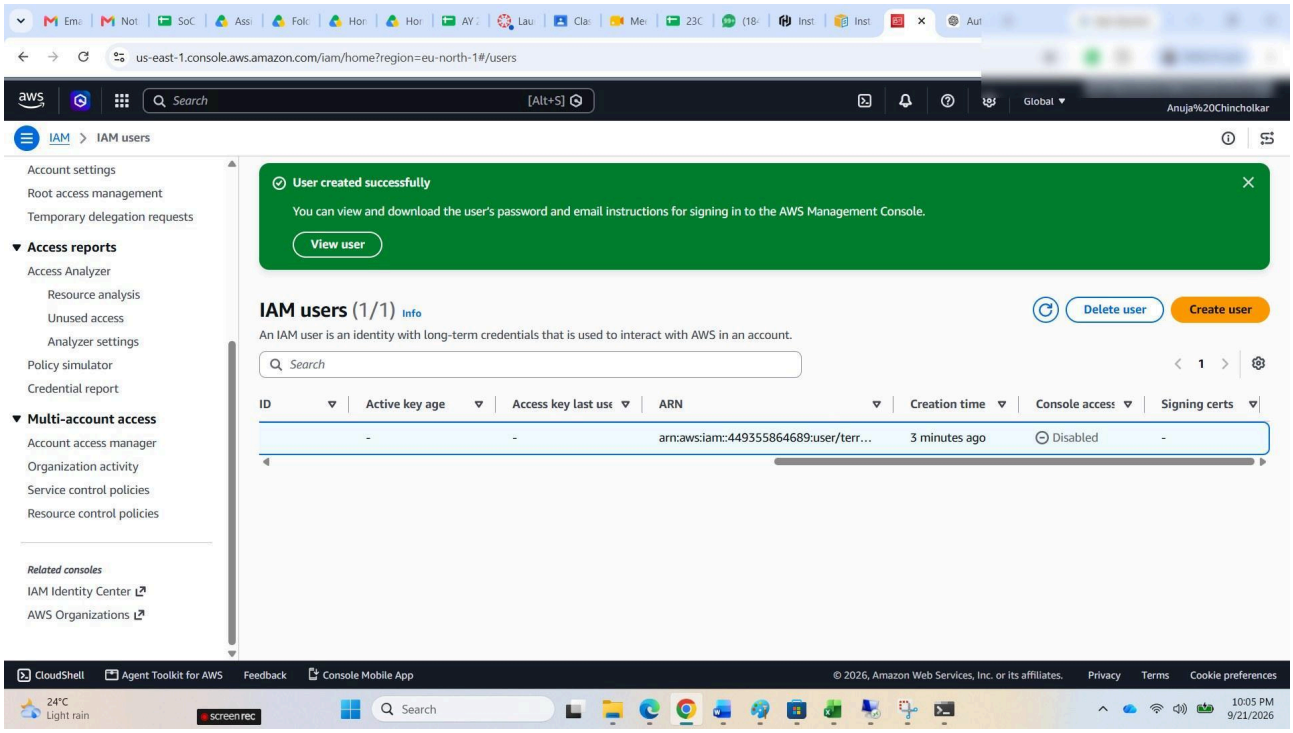
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\Admin\terraform-network>
```

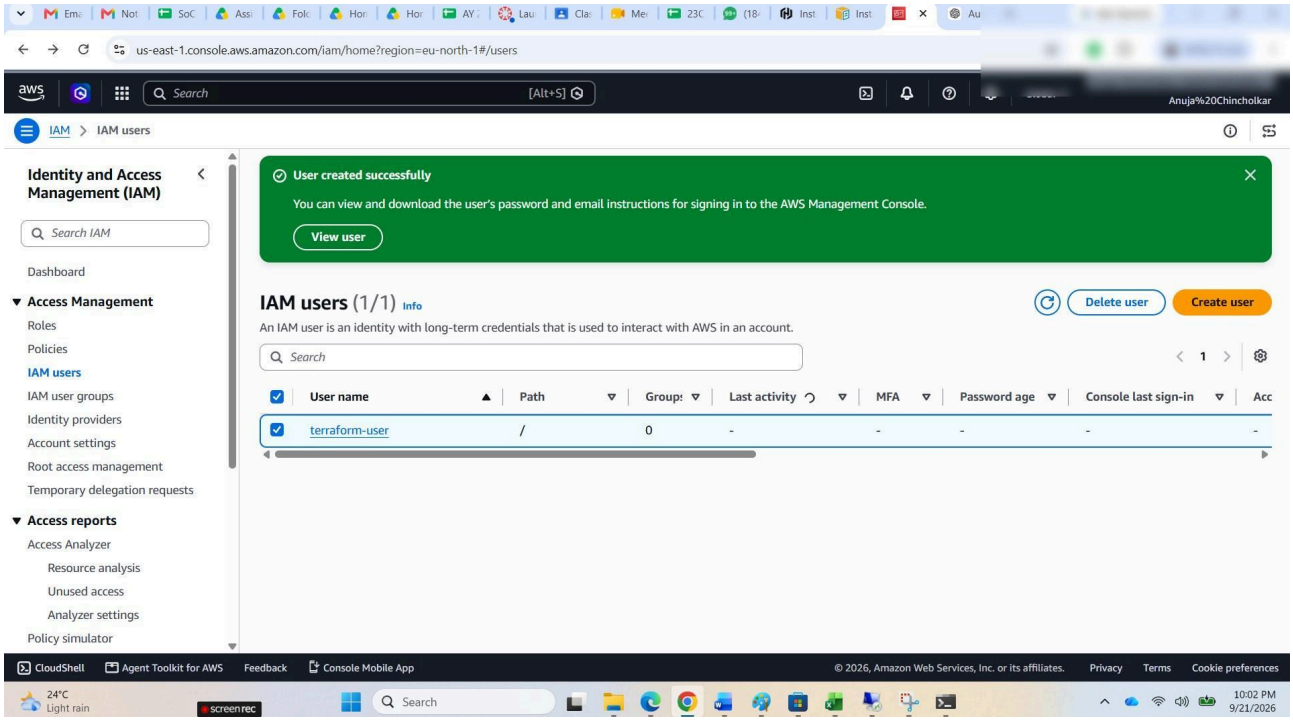
Screenshot 8



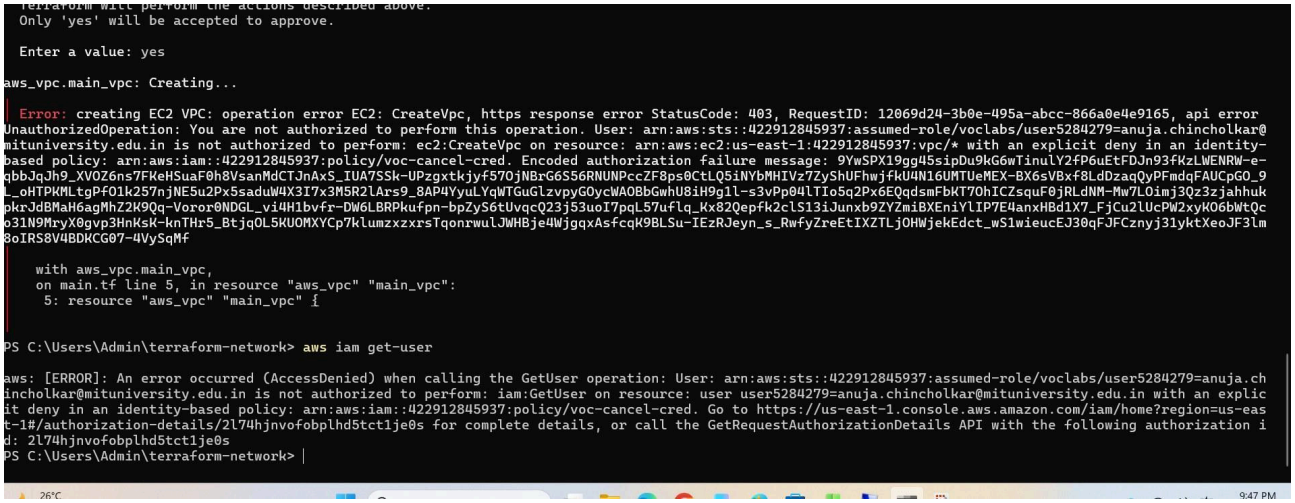
Screenshot 9



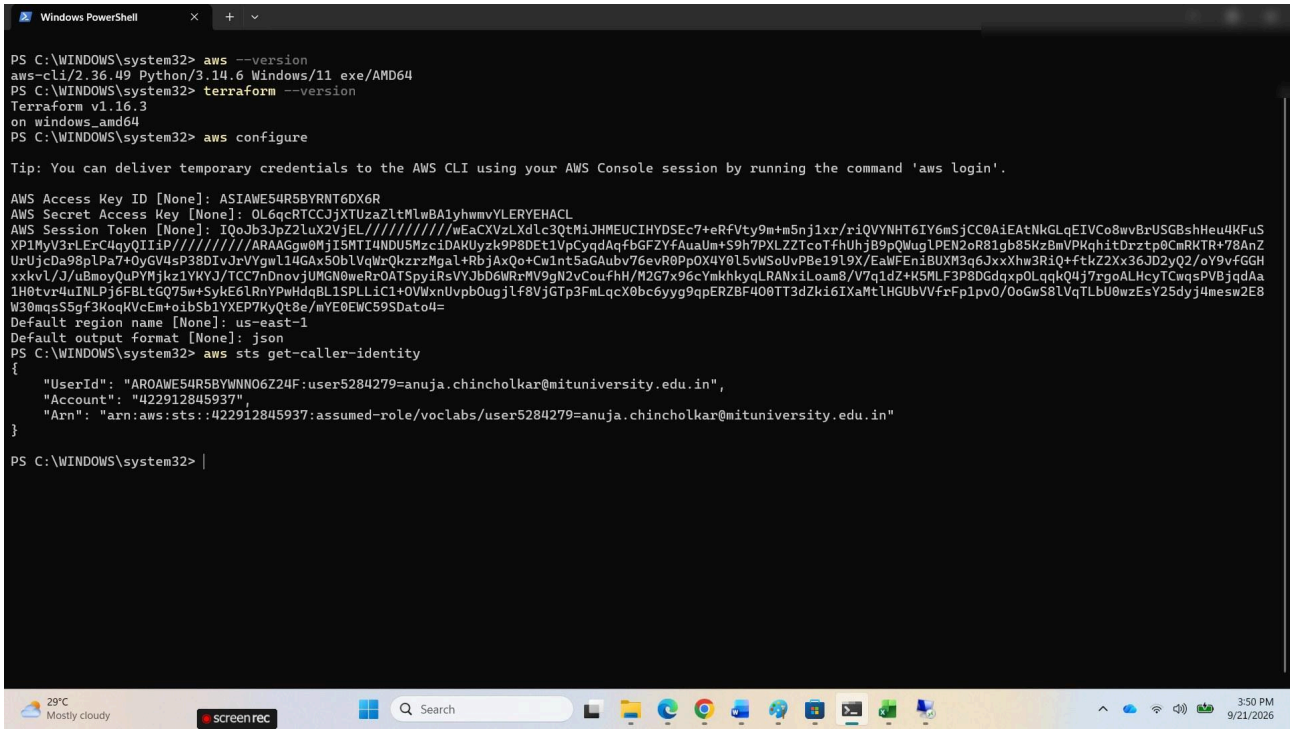
Screenshot 10



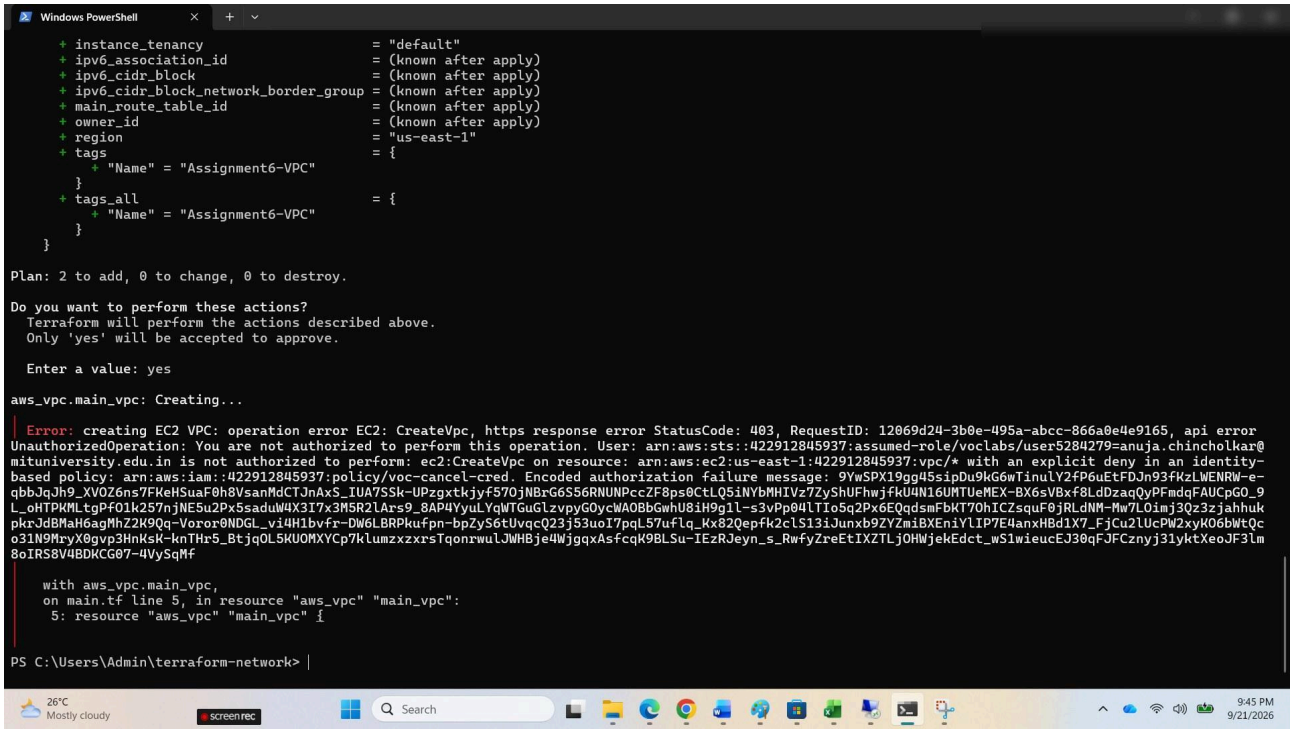
Screenshot 11



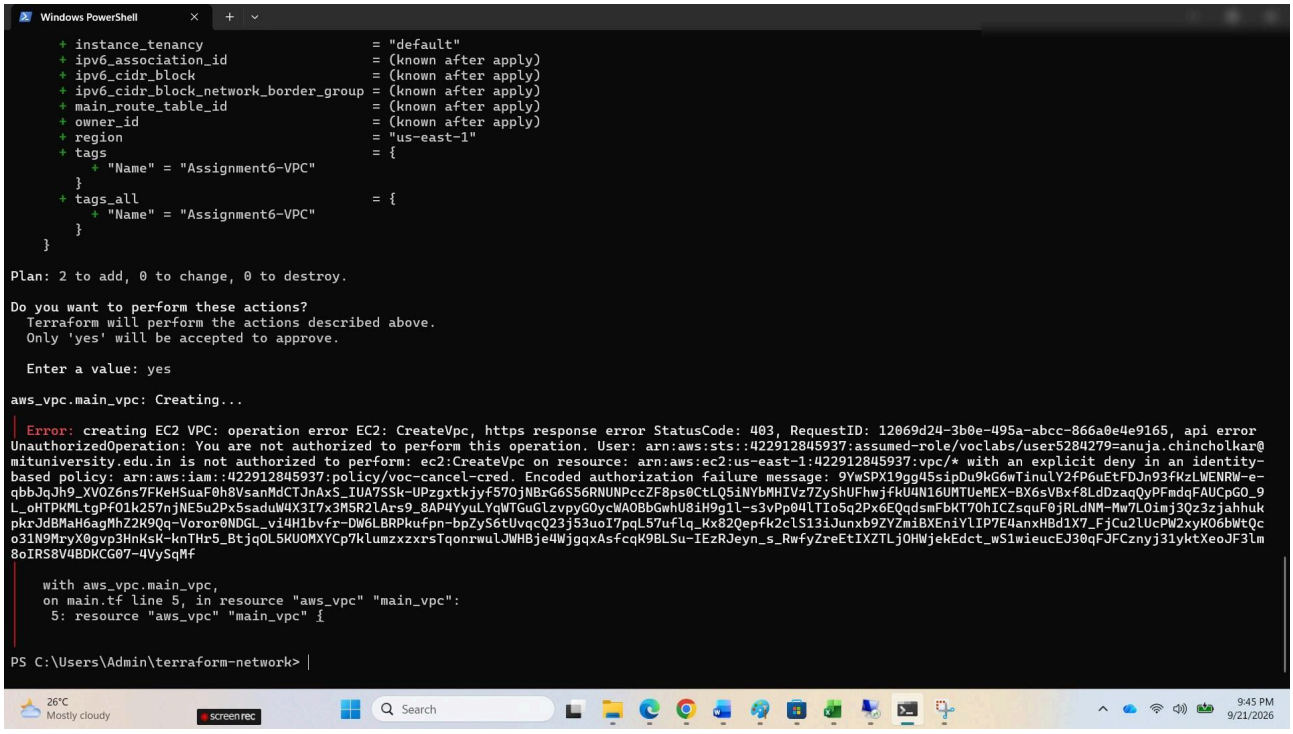
Screenshot 12



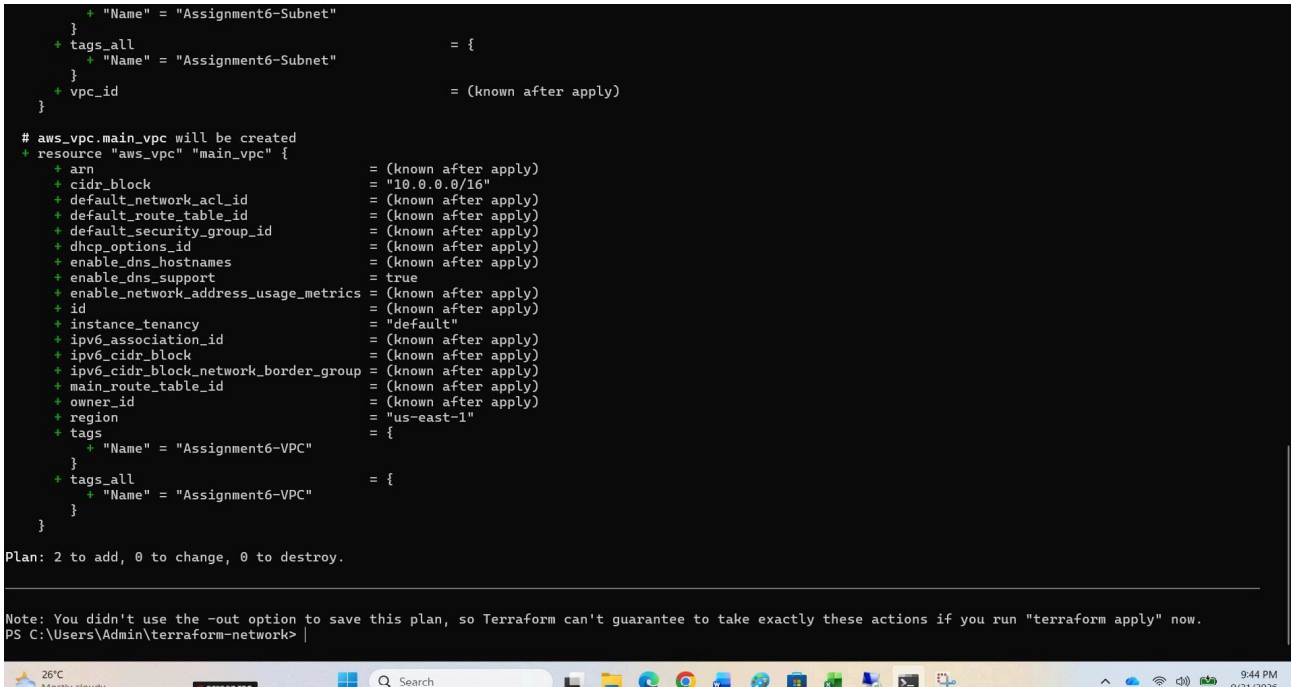
Screenshot 13



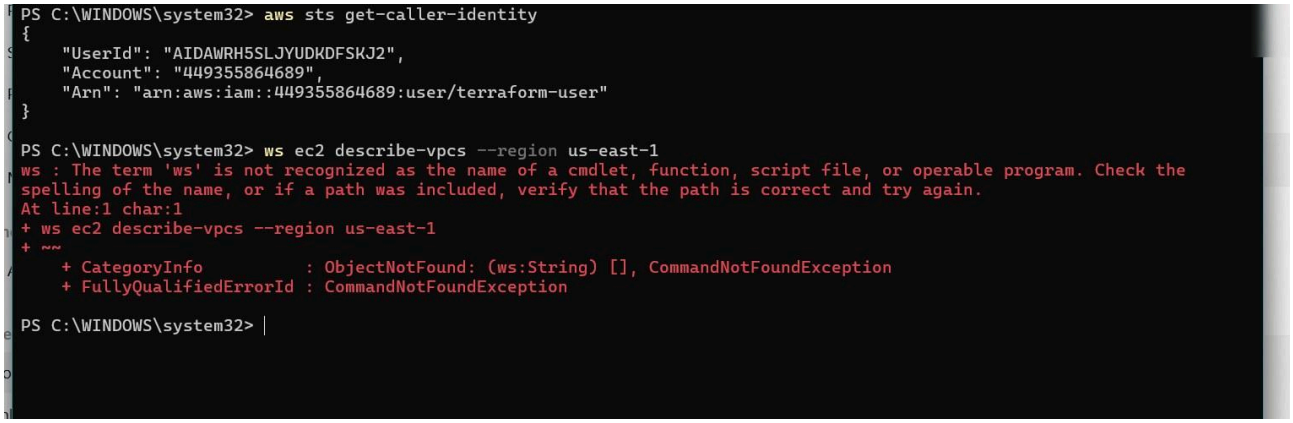
Screenshot 14



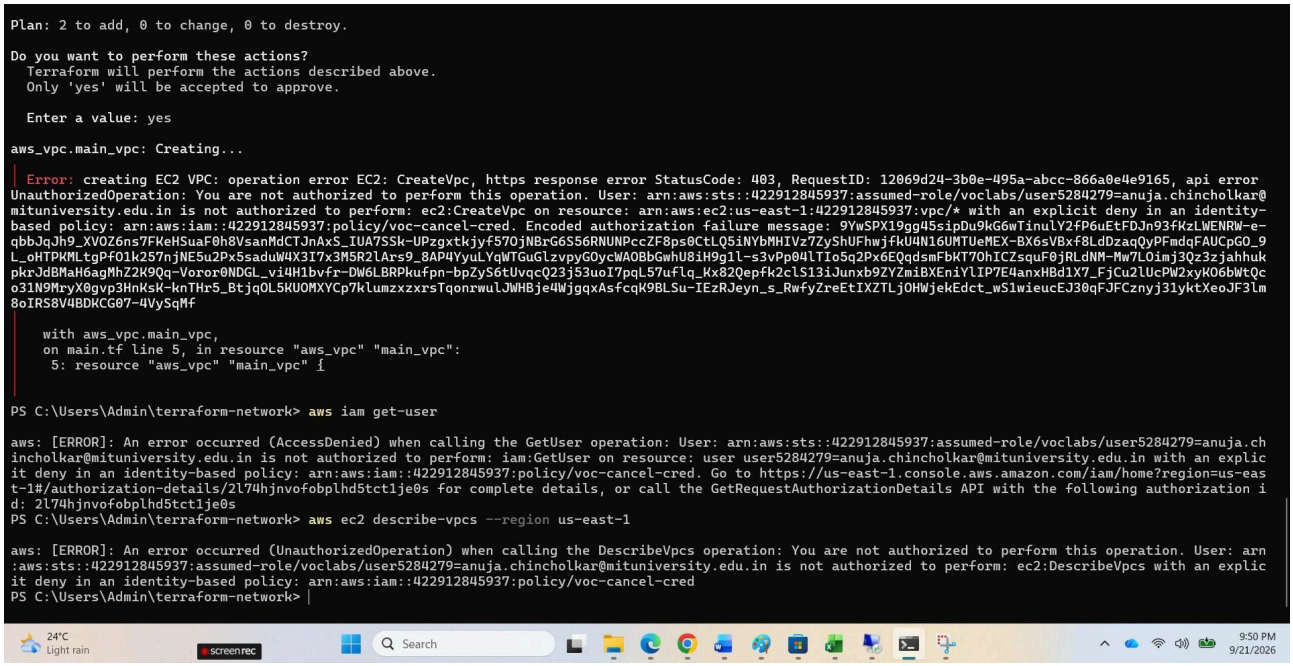
Screenshot 15



Screenshot 16



Screenshot 17


Plan: 2 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_vpc.main_vpc: Creating...
Error: creating EC2 VPC: operation error EC2: CreateVpc, https response error StatusCode: 403, RequestID: 12069d24-3b0e-495a-abcc-866a0e4e9165, api error UnauthorizedOperation: You are not authorized to perform this operation. User: arn:aws:sts::422912845937:assumed-role/voclabs/user5284279=anuja.chincholkar@mituniversity.edu.in is not authorized to perform: ec2:CreateVpc on resource: arn:aws:ec2:us-east-1:422912845937:vpc/* with an explicit deny in an identity-based policy: arn:aws:iam::422912845937:policy/voc-cancel-cred. Encoded authorization failure message: 9YwSPX19gg45sipDu9kG6wTinulY2fP6uEtFDJn93fKzLWENRW-e-qbbJqJh9_XVOZ6ns7FKeHSuaF0h8VsanMdCTJnAxS_IUA7SSk-UPzgxtkjyf57OjNBrG6S56RNUNPccZF8ps0CtLQ5iNYbMHIVz7ZyShUFhwjfkU4N16UMTUeMEX-BX6sVBxf8LdDzaqQyPFmdqFAUCpGO_9L_oHTPKMLtgPfO1k257njNE5u2Px5saduW4X3I7x3M5R2lArs9_8AP4YyuLYqWTGuGlzvpyGOycWAOBbGwhU8iH9g1l-s3vPp04lTIo5q2Px6EQqdsmFbKT7OhICZsquF0jRLdNM-Mw7LOimj3Qz3zjahhukpkrJdBMaH6agMhZ2K9Qq-Voror0NDGL_vi4H1bvfr-DW6LBRPkufpn-bpZyS6tUvqcQ23j53uoI7pqL57uflq_Kx82Qepfk2clS13iJunxb9ZYZmiBXEniYlIP7E4anxHBd1X7_FjCu2lUcPW2xyKO6bWtQco31N9MryX0gvp3HnKsK-knTHr5_BtjqOL5KUOMXYCp7klumzxzxrsTqonrwulJWHBje4WjgqxAsfcqK9BLSu-IEzRJeyn_s_RwfyZreEtIXZTLjOHWjekEdct_wS1wieucEJ30qFJFCznyj31yktXeoJF3lm8oIRS8V4BDKCG07-4VySqMf
with aws_vpc.main_vpc,
on main.tf line 5, in resource "aws_vpc" "main_vpc":
5: resource "aws_vpc" "main_vpc" {
PS C:\Users\Admin\terraform-network> aws iam get-user
aws: [ERROR]: An error occurred (AccessDenied) when calling the GetUser operation: User: arn:aws:sts::422912845937:assumed-role/voclabs/user5284279=anuja.chincholkar@mituniversity.edu.in is not authorized to perform: iam:GetUser on resource: user user5284279=anuja.chincholkar@mituniversity.edu.in with an explicit deny in an identity-based policy: arn:aws:iam::422912845937:policy/voc-cancel-cred. Go to https://us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/authorization-details/2l74hjnvofobplhd5tct1je0s for complete details, or call the GetRequestAuthorizationDetails API with the following authorization id: 2l74hjnvofobplhd5tct1je0s
PS C:\Users\Admin\terraform-network> aws ec2 describe-vpcs --region us-east-1
aws: [ERROR]: An error occurred (UnauthorizedOperation) when calling the DescribeVpcs operation: You are not authorized to perform this operation. User: arn:aws:sts::422912845937:assumed-role/voclabs/user5284279=anuja.chincholkar@mituniversity.edu.in is not authorized to perform: ec2:DescribeVpcs with an explicit deny in an identity-based policy: arn:aws:iam::422912845937:policy/voc-cancel-cred
PS C:\Users\Admin\terraform-network>
24°C
Light rain
screenrec
Search
9:50 PM
9/21/2026

20. Official Documentation

AWS CLI installation: https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html

Terraform installation: https://developer.hashicorp.com/terraform/install

Terraform on AWS tutorial: https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli

AWS IAM access keys:
https://docs.aws.amazon.com/IAM/latest/UserGuide/access-keys-admin-managed.html

End of Guide